Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

ОТЧЁТ

по лабораторной работе № 3
по дисциплине «Проектирование систем на кристалле»
«Анализ выполнения инструкции DIV в конвейере SCR1»
Вариант 9

Выполнила:
Пронина Анастасия Ильинична
Группа: P4130

Санкт-Петербург
2026

1. Цель и задание

Изучить выполнение инструкции знакового целочисленного деления DIV в конвейере SCR1. Добавить несинтезируемый наблюдатель, выводящий атрибуты команды, и исследовать её выполнение по временной диаграмме.

Вариант 9: инструкция `div`; отладочная информация — значения атрибутов команды. Рабочая ветка — `lab3_scr1_pipe_analys`.

2. Окружение и запуск

Использовано Docker-окружение проекта, Verilator 5.030, интерфейс AHB и конфигурация MAX (RV32IMC). Для анализа записана VCD-трасса с текстовой трассировкой.

В `sim/tests/riscv_isa/rv32_tests.inc` оставлен единственный тест:

```
rv32_isa_tests := isa/rv32um/div.S
```

Команда запуска внутри контейнера, из `/work/scr1`:

```
make run_verilator_wf CFG=MAX BUS=AHB TARGETS=riscv_isa TRACE=1
```

Результаты находятся в `build/verilator_wf_AHB_MAX_imc_IPIC_1_TCM_1_VIRQ_1_TRACE_1/`. Файлы `div.dump`, `simx.vcd` и `test_results.txt` использованы для проверки адресов, времён и результата теста.

2.1. Последовательность выполненных действий

1. Подготовлена рабочая ветка. Работа ведётся в отдельной ветке `lab3_scr1_pipe_analys` репозитория SCR1. Её исходный коммит — `ebb5e35`. Удалённый `origin` настроен на личный репозиторий https://github.com/ProshkaAnastasia/scr1; для отправки используется SSH. Реализация лабораторной зафиксирована коммитом `60646bb`
   (`Add simulation`); ветка с реализацией отправлена в личный репозиторий.
2. Выбран тест по варианту. Из списка ISA-тестов удалены остальные тесты и условные добавления наборов M/C. Оставлена только строка с `isa/rv32um/div.S`. Выбор `CFG=MAX` обеспечивает поддержку расширения M, необходимого для DIV; редактировать декодер для включения деления не требовалось.
3. Получена первоначальная диаграмма. Симуляция запущена с целью `run_verilator_wf` и параметром `TRACE=1`. Найден файл `simx.vcd`, открытый в GTKWave на macOS; сама модель выполнялась в Docker.
4. Изучен путь команды в исходниках. В IDU найдена ветвь регистровых операций расширения M, выбирающая `SCR1_IALU_CMD_DIV`. В EXU прослежены приём команды, чтение MPRF, подключение IALU, готовность и запись результата. В `div.dump` выбран адрес первой DIV — `0x2A8`.
5. Добавлен отладочный модуль. Создан `src/tb/scr1_tb_log_cmd.sv`. Его экземпляр `i_log_cmd` подключён в AHB-тестбенче, а исходник добавлен в `src/ahb_tb.files`. Модуль наблюдает завершение DIV по действующей команде и разрешению записи. Процессор не создаётся повторно: используются сигналы уже существующего экземпляра EXU.
6. Выполнена повторная симуляция с наблюдателем. После включения нового модуля сборка и запуск повторены. Получены девять сообщений `[DIV]`, результат `div.hex: OK, PASS` и итог `1/1 tests passed`. Адреса сообщений сопоставлены с дизассемблированием; значения результатов проверены, включая особые случаи.
7. Настроены сигналы GTKWave. Такт взят с верхнего уровня или из EXU, текущий PC — из EXU (`pc_i` наблюдателя), а четыре сигнала памяти инструкций — из `i_top.i_imem_ahb`. Дополнительно выведены значения операндов, номера регистров, достоверность команды, разрешение записи и результат. Выбран интервал, включающий поступление DIV в EXU и запись частного.

8. Сохранены материалы анализа. Скриншот GTKWave сохранён в `diagram.png`, текстовый вывод — в `div_debug.log`, результат теста — в `test_results.txt`. Скриншот включён в данный отчёт с обрезкой; дополнительно приведены поясняющие рисунки из той же VCD-трассы.

3. Изменения в проекте

Файл	Изменение
`sim/tests/riscv_isa/rv32_tests.inc`	Выбран только тест `div.S`.
`src/tb/scr1_tb_log_cmd.sv`	Добавлен наблюдатель исполнения DIV.
`src/ahb_tb.files`	Наблюдатель включён в список исходников.
`src/tb/scr1_top_tb_ahb.sv`	Создан экземпляр наблюдателя и подключены сигналы EXU.

Процессорное RTL не изменялось для добавления наблюдателя. Код вывода размещён в тестовом окружении и ограничен директивами `synthesis translate_off/on`.

4. Устройство наблюдателя

Модуль не загружает программу и не выполняет деление. Он получает сигналы экземпляра `i_top.i_core_top.i_pipe_` и наблюдает завершение инструкции.

Вход наблюдателя	Сигнал EXU	Смысл
`clk`, `rst_n`	`clk`, `rst_n`	Такт и сброс.
`valid_i`	`exu_queue_vd`	Текущая команда достоверна.
`cmd_i`	`exu_queue`	Декодированная операция и номера регистров.
`pc_i`	`pc_curr_ff`	PC стадии исполнения.
`operand1_i`, `operand2_i`	`ialu_main_op1`, `ialu_main_op2`	Делимое и делитель.
`writeback_i`	`exu2mprf_w_req_o`	Разрешена запись результата.
`result_i`	`exu2mprf_rd_data_o`	Данные для записи в регистр.

На положительном фронте такта проверяется:

```
rst_n && valid_i && writeback_i
    && (cmd_i.ialu_cmd == SCR1_IALU_CMD_DIV)
```

Это позволяет печатать одно сообщение при записи результата каждой завершённой DIV, вместо повторения на каждом такте многотактного вычисления. Выводятся PC, номера `rs1`, `rs2`, `rd`, значения операндов и результат. Приведение `$signed` обеспечивает знаковое десятичное представление.

`valid_i` не означает завершение команды: он остаётся установленным во время исполнения. `writeback_i` означает разрешение записи на фронте. При последовательности однотактных команд разрешение может оставаться единицей несколько тактов, обслуживая разные инструкции.

5. Прохождение DIV через конвейер

1. IFU выбирает машинное слово из памяти инструкций и передаёт его IDU.
2. IDU распознаёт `opcode = 0110011`, `funct7 = 0000001`, `funct3 = 100` как DIV. Он формирует `SCR1_IALU_CMD_DIV`, выбирает два регистровых операнда и результат IALU для записи в `rd`.
3. При готовности EXU команда сохраняется в `exu_queue`. Из MPRF читаются значения регистров-источников. В используемой реализации MPRF чтение комбинационное, запись — по фронту.

4. IALU выполняет итерационное деление. Пока результат не готов, EXU удерживает текущую команду и не принимает следующую в свой буфер.
5. При готовности частного разрешается запись. На положительном фронте MPRF фиксирует результат; при обычном продолжении PC увеличивается на 4.

Для первой проверки исходник содержит `TEST_RR_OP(2, div, 3, 20, 6)`. В дизассемблировании:

```
2a0: 01400093  li  ra,20
2a4: 00600113  li  sp,6
2a8: 0220c1b3  div gp,ra,sp
2ac: 00300e93  li  t4,3
2b0: 00200e13  li  t3,2
2b4: 0dd19463  bne gp,t4,37c <fail>
```

Здесь `ra`=x1, `sp`=x2, `gp`=x3, `t4`=x29, `t3`=x28. Деление записывает частное в x3. Затем тест сравнивает x3 с эталоном 3; при несовпадении выполняется переход на `fail`.

6. Временные диаграммы

Ниже приведён скриншот GTKWave и поясняющие диаграммы, построенные непосредственно по изменениям сигналов `simx.vcd`. Скриншот обрезан средствами LaTeX: сохранены названия сигналов, шкала времени и графики; дерево модулей и пустая нижняя область скрыты. Исходный файл `diagram.png` не изменён.

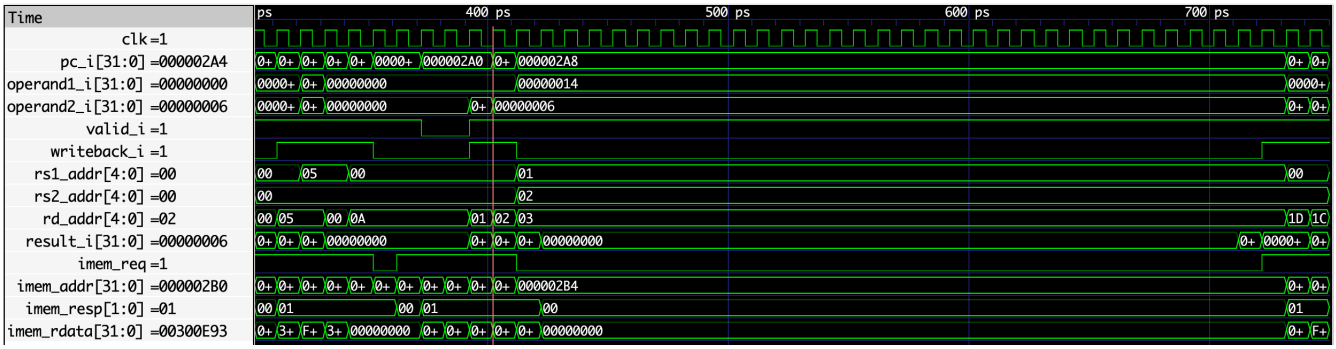


Рис. 1: Скриншот GTKWave: исполнение DIV по адресу 0x2A8, операнды 0x14 и 0x06, регистры x1, x2 и x3.

На скриншоте значения после знака равенства слева от графиков относятся к позиции маркера, а не ко всему интервалу. Команду DIV следует идентифицировать по участку `pc_i = 0x2A8` при `valid_i = 1`; её результат учитывается при разрешённой записи.

Следующие поясняющие диаграммы сохраняют реальные времена и значения той же трассы. Адреса и данные инструкций показаны в шестнадцатеричной системе, операнды и арифметический результат — в знаковой десятичной; время — в ps.

Включены обязательные сигналы задания: `clk`, текущий PC стадии исполнения (в наблюдателе называется `pc_i`), `imem_req`, `imem_addr`, `imem_resp`, `imem_rdata`. Дополнительно показаны операнды, готовность IALU, достоверность команды, результат и разрешение записи.

Время, ps	Наблюдаемое событие
412	PC исполнения принимает значение 0x2A8. Операнды равны 0x14 (20) и 0x06 (6). Готовность и разрешение записи опускаются.
412–722	Команда удерживается в EXU, вычисление продолжается. Промежуточное значение результата нельзя считать готовым частным.
722	IALU выдаёт готовность. Результат равен 3, запись разрешена.

Время, ps	Наблюдаемое событие
732	На положительном фронте число 3 записывается в x3. Наблюдатель печатает сообщение. PC переходит к 0x2AC.

Период такта в трассе — 10 ps. Интервал от приёма команды до фронта записи составляет 732 − 412 = 320 ps, то есть 32 тактовых периода для выбранной операции. Это модельное время RTL-симуляции, а не оценка допустимой частоты реального кристалла.

Выборка инструкций опережает исполнение: при PC исполнения 0x2A8 интерфейс памяти может обращаться к последующим адресам. В частности, код 0x00200E13 соответствует `li x28, 2` по адресу 0x2B0, а не DIV.

Данные `imem_rdata` трактуются как достоверная инструкция при успешном ответе `imem_resp = 01`. При `imem_resp = 00` готового ответа нет; нулевые данные на шине в это время не означают выполнение нулевой инструкции. Значение `imem_addr` без принятого запроса также не означает новую выборку.

7. Результаты проверки

Симуляция завершилась результатом 1/1 tests passed, `div.hex — OK, PASS`. Наблюдатель вывел девять операций, полный вывод сохранён в `report/div_debug.log`.

PC	Делимое	Делитель	Частное
0x2A8	20	6	3
0x2C0	-20	6	-3
0x2D8	20	-6	-3
0x2F0	-20	-6	3
0x308	-2147483648	1	-2147483648
0x320	-2147483648	-1	-2147483648
0x338	-2147483648	0	-1
0x350	1	0	-1
0x368	0	0	-1

Проверены разные сочетания знаков, граничное отрицательное значение и деление на ноль. Для DIV деление на ноль даёт все единицы в результате (−1 при знаковом отображении), а переполнение −2147483648 / −1 даёт −2147483648. Эти случаи не вызывают исключения деления на ноль или переполнения.

Пример фактической строки вывода наблюдателя (первая операция):

```
[DIV] PC=0x000002a8 rs1=x1 rs2=x2 rd=x3 operand1=20 operand2=6 result=3
```

Номер регистра результата `rd=x3` и значение `result=3` здесь совпадают численно случайно: первое указывает место записи, второе является вычисленным частным. Вывод сделан на фронте записи результата, а не при выборке машинного слова из памяти.

Наблюдатель выводит данные, а правильность результата проверяет программа ISA-теста. Адреса всех девяти сообщений сопоставлены с инструкциями DIV в `div.dump`.

8. Вывод

Исследован путь DIV от декодирования до записи частного в регистр. Добавлен несинтезируемый наблюдатель, выводящий атрибуты завершённой инструкции. Для операции 20 / 6 подтверждены PC 0x2A8, источники x1 и x2, результат 3 в x3 и длительность пребывания команды в EXU 32 периода такта. Тест завершился успешно.

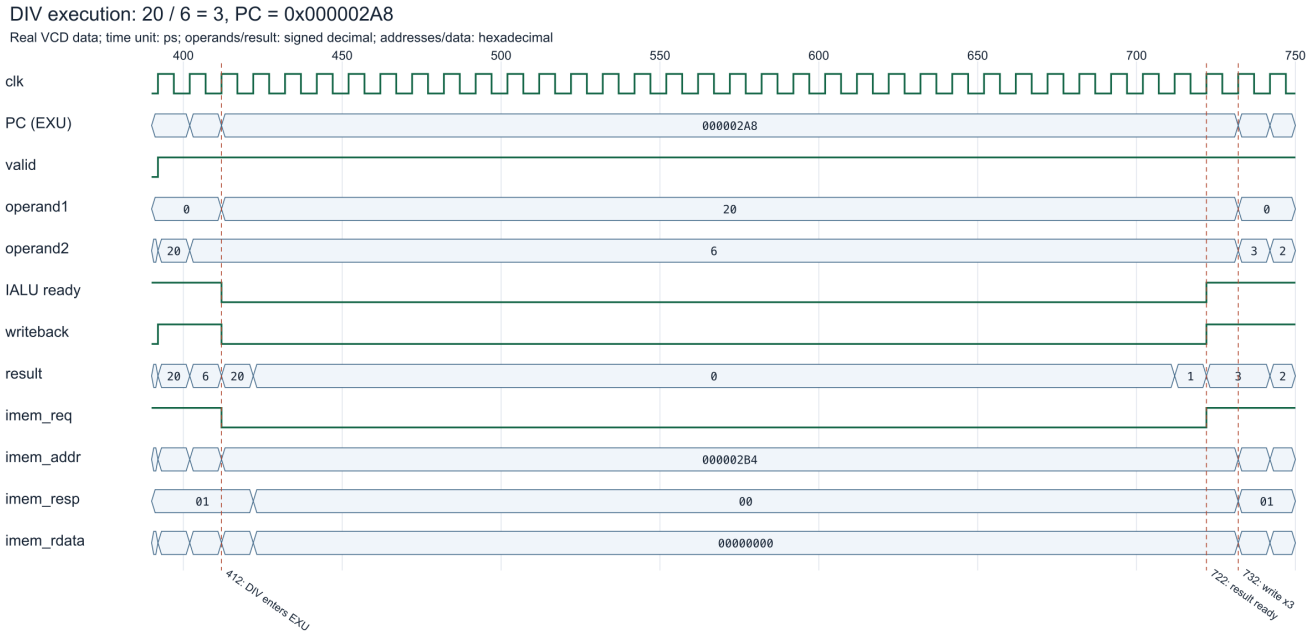


Рис. 2: Исполнение первой DIV и запись результата.

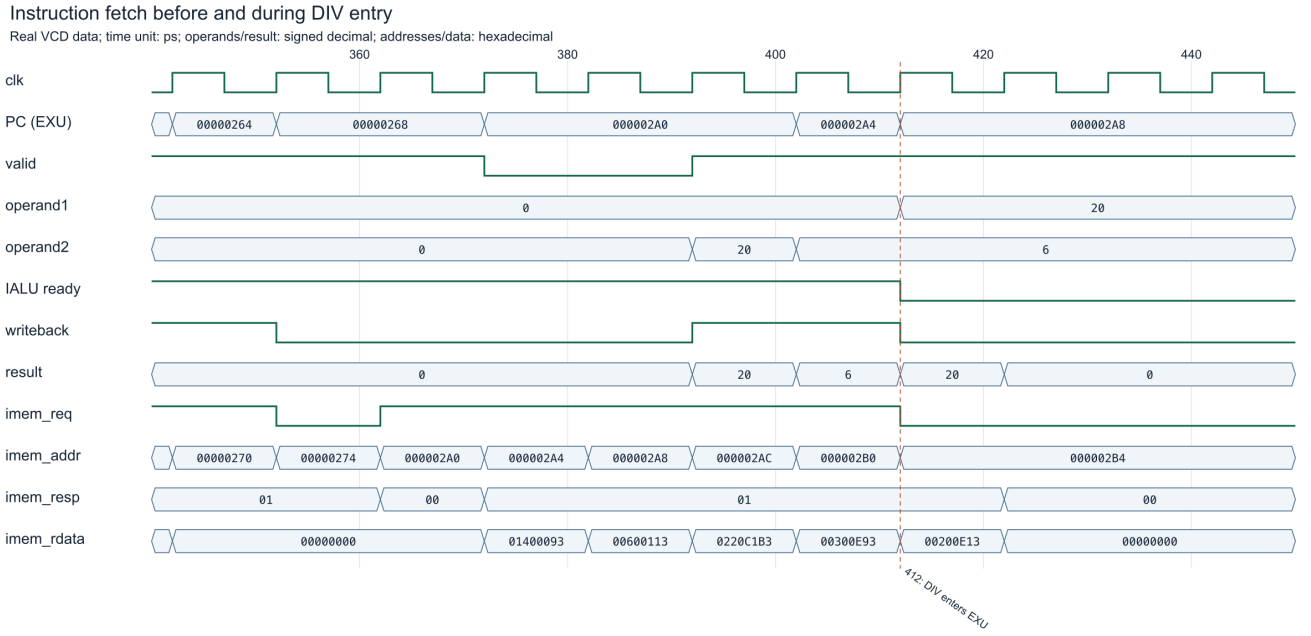


Рис. 3: Интерфейс выборки в окрестности поступления DIV в EXU.

9. Использованные материалы

- Методические указания «Лабораторная работа 3», пункты задания и вариант 9.
- Исходные файлы SCR1: `scr1_pipe_idu.sv`, `scr1_pipe_exu.sv`, `scr1_pipe_ialu.sv`, `scr1_pipe_mprf.sv`.
- Результаты текущего запуска: `div.dump`, `simx.vcd`, `test_results.txt`, `report/div_debug.log`.

10. Итог выполнения задания

1. Создана отдельная ветка `lab3_scr1_pipe_analys`.
2. Выбран и запущен единственный тест `isa/rv32um/div.S`. Симуляция завершилась результатом `PASS`.
3. Исследовано выполнение DIV в конвейере. На диаграмму выведены обязательные сигналы и дополнительные данные для анализа операндов, готовности и записи результата.
4. Добавлен несинтезируемый наблюдатель `scr1_tb_log_cmd`. Его вывод содержит адрес команды, номера регистров, значения операндов и вычисленное частное. Получено девять сообщений о завершённых DIV.
5. В папке `report/` подготовлены отчёт, скриншот GTKWave, поясняющие диаграммы, текстовый вывод наблюдателя и результат теста.
6. Изменения исходников зафиксированы коммитом `60646bb` (`Add simulation`); ветка с реализацией отправлена в личный репозиторий GitHub.

Для повторной сборки отчёта из папки `report/` используется:

```
xelatex lab3_report.tex
```

Исходник LaTeX использует `diagram.png` и изображения из `assets/`. Папка сгенерированной модели `build/` для чтения отчёта не требуется.